

Network Visualiser — Project State and Decision Log

A re-onboarding document: the workflow as built, and every engineering decision behind it

Amr Shariff

17 August 2026

Contents

1	Purpose of this document	3
2	What the project is	3
2.1	Repository and environment	3
3	The pipeline as it stands today	4
3.1	Key numbers, for orientation	4
4	Decision log — Phase 0 and Phase 1	5
4.1	Do not scrape LinkedIn	5
4.2	Build a synthetic generator first	5
4.3	Sentence-transformers over TF-IDF or a hosted embedding API	5
4.4	Cosine similarity throughout	5
4.5	PageRank over eigenvector centrality	6
4.6	Threshold by target degree, not by intuition	6
5	Phase 1.5 — the pivot, and the load-bearing finding	7
5.1	What broke	7
5.2	The reframe	7
5.3	The classification ceiling	7
5.4	The bake-off	7
5.5	Ruled out permanently, with evidence	8
5.6	Coverage-first with visible uncertainty	8
5.7	Scope limit: major group only	9
6	Decision log — Phase 2 (dashboard)	10
6.1	Selection is list-driven, not click-driven	10
6.2	The correction overlay (<code>state.py</code>)	10
6.3	A fixed 23-colour palette	10
6.4	Faded-node highlighting	10
6.5	Edge overlay — built, then reverted	11
6.6	Day-0 data reconciliation	11
7	Environment and tooling issues encountered	12

7.1	The OneDrive migration	12
7.2	The wrong-venv trap	12
7.3	The zombie Streamlit server	12
7.4	The <code>@st.cache_data</code> trap	12
7.5	The <code>requirements.txt</code> placeholder	13
7.6	Plotly Scatter3d quirks	13
7.7	Standing commands	13
8	Data artefacts and privacy posture	14
8.1	Cross-cutting rules	14
9	Current board and what remains	15
9.1	Phase 2 — MVP dashboard (estimated 8–11 focused days)	15
9.2	Phase 3 — Team composition and network insight (5–7 days)	15
9.3	Phase 4 — Polish, narrative, public launch (4–5 days)	15
10	Open decisions and parked items	16
10.1	The 2D / 3D toggle — parked to Phase 4	16
10.2	Taxonomy skills on profile cards — unresolved	16
10.3	Persistent in-scene node label	16
10.4	Stage 2 productionisation — named, not built	16
11	The next task in full	17
11.1	Amended definition of done	17

Correction note — 21 September 2026 (D-42). The needs-review figure in this document, 168 of 430 (39.1%), was measured while a classifier defect was caching failed API batches as abstentions. How many of the 168 were affected cannot be recovered, because the cache does not timestamp its entries, so read the figure as an upper bound. At 442 rows the corrected figure is **128 (29.0%)**. The decisions and reasoning below are unaffected. The document is left as written. See `docs/d42_failure_record.md` for the cause, the evidence and the fix.

Correction note — 29 September 2026 (D-69). Of the three convergent measurements of the classification ceiling, only the third — Config E’s 63.5% precision on the blind 100 — involves the classifier, and it is a single run. The classifier is not deterministic: five builds of an identical 442-person synthetic file, cache cleared between each, gave needs-review shares from 21.9% to 28.1%, with about 15% of titles receiving a different group between the one pair diffed. The blind 100’s own spread has not been measured and may be wider at a hundred titles. The first two measurements are human and are unaffected, and the convergence argument stands on them. The document below is left as written. See `docs/defect_register.md`, D-69.

1 Purpose of this document

This is a re-onboarding document. It records where the project stands, how the workflow actually operates today, and — most importantly — *why* each significant decision was made, including the ones that reversed earlier decisions.

It is written to be read cold, after time away. Where a decision was driven by a measurement, the measurement is quoted. Where a decision was reversed, both the original reasoning and the reason for reversal are kept, because the reversals carry more information than the successes.

Currency note. The last recorded working session was 27 July 2026. If work has happened since, this document is stale from that date onward. Verify with `git log --oneline -20` before trusting the board in Section 9.

2 What the project is

An interactive dashboard that ingests a LinkedIn *Connections* CSV export, classifies each connection’s job title into an occupational taxonomy, renders the network in 3D, and lets the user search it, correct it, and query it in natural language for team-building suggestions.

Primary success criterion: correct behaviour on real, arbitrary LinkedIn data. Not methodological novelty, not portfolio optics. The tool has to work on a stranger’s network, not just one curated example.

Deployment model — Version A. Users upload their own export; it is processed ephemerally and never persisted server-side. This was the decision that resolved both the LinkedIn Terms of Service problem and the UK GDPR problem in one move, and it has a hard consequence that shapes everything downstream: *the pipeline must generalise*. Nothing may be hardcoded to one network’s shape.

Secondary beneficiary: a UCL MSc (Machine Learning with Data Science) application. This is a beneficiary, not a driver. Where the two conflict — and they did, notably over HDBSCAN — product correctness wins.

2.1 Repository and environment

Repository	<code>github.com/amrxshariff/network-visualiser</code> (private)
Local path	<code>C:\dev\network-visualiser</code>
Stack	Python, <code>.venv</code> , PowerShell / Windows, VS Code
Dashboard	Streamlit + Plotly
Taxonomy	O*NET 30.3 Database (CC BY 4.0, USDOL/ETA)

3 The pipeline as it stands today

The system is a linear pipeline with a dashboard wrapped around the final stages. Each stage has a clean input and output, which is what makes stages individually replaceable — and two of them have already been replaced.

#	Stage	Module	Input	Output
1	Ingestion	<code>linkedin.py</code> , <code>ingestion.py</code>	Connections.csv	Person objects
2	Classification	<code>claude_classifier.py</code> <code>onet.py</code>	Job titles	SOC major group or abstain
3	Embedding	<code>embeddings.py</code>	<code>profile_text</code>	384-dim vectors
4	Projection	<code>projection.py</code>	Vectors	UMAP coordinates (2D + 3D sidecar)
5	Graph	<code>graph.py</code>	Vectors	Cosine-similarity graph
6	Centrality	<code>centrality.py</code>	Graph	Canonical node table
7	Dashboard	<code>dashboard/</code>	Node table	Streamlit + Plotly UI

The canonical handoff is `data/graph/network_nodes.csv` — 430 rows carrying identity, SOC group, UMAP coordinates, centrality metrics, and the `is_uncertain` flag. Everything in Phase 2 and beyond reads from this table. It is the contract between the engine and the interface.

3.1 Key numbers, for orientation

Measure	Value
People in the network	430
Distinct job titles	~371
Classified	262 (60.9%)
Flagged “Needs review”	168 (39.1%)
SOC major groups in taxonomy	23
Embedding dimensionality	384 (<code>all-MiniLM-L6-v2</code>)
Graph threshold τ	0.7710
Graph modularity Q	0.5224
Connected components / isolates	13 / 4
UMAP trustworthiness ($k = 5$)	0.9840

The uncertain count is **168, not 169**. The earlier figure of 169 predates the Day-0 reconciliation, during which one boundary-case node moved from uncertain to classified. The number is derived live from the table and is hardcoded nowhere.

4 Decision log — Phase 0 and Phase 1

4.1 Do not scrape LinkedIn

Decision: abandon the original concept of an agent that logs into LinkedIn and crawls each connection’s profile. Use the official data export instead.

Reasoning: three independent reasons, any one sufficient. LinkedIn’s User Agreement prohibits automated collection and detects it. Under UK GDPR every connection is a data subject and profiling them without a lawful basis is a genuine compliance exposure, not a technicality. And a scraper is a liability on a portfolio, not an asset.

Consequence: the ingestion layer was designed so that swapping the data source is a configuration change (`config.DATA_SOURCE`), not a rewrite. This turned out to matter enormously — it is what let the project run synthetic and real data through identical downstream code.

4.2 Build a synthetic generator first

Decision: `synthetic.py` produces ~80 realistic fake profiles across 11 archetypes, with deliberate noise (profiles straddling two archetypes) and three genuine outliers.

Reasoning: it unblocks every later stage without touching real data, and it lets the ML engine be proven before any UI exists — so you are never debugging the model and the interface simultaneously.

What it cost, discovered later: the generator encoded *our own assumptions* about which roles imply which skills. Any classifier trained or evaluated against it can only recover those assumptions. This was named the **generator-inversion ceiling**, and it is why the synthetic archetype enrichment was eventually superseded entirely (Section 5).

4.3 Sentence-transformers over TF-IDF or a hosted embedding API

Decision: `all-MiniLM-L6-v2`, 384 dimensions, cosine similarity.

Reasoning: captures semantic proximity that bag-of-words cannot (“ML Engineer” $\approx$ “Data Scientist”), runs locally at no cost, no API key, and small enough to deploy anywhere. TF-IDF is still used, appropriately, for cluster labelling — a different job.

4.4 Cosine similarity throughout

Decision: cosine as the metric at every stage — clustering, projection, and graph edges.

Reasoning: consistency. Sentence-transformer magnitude carries little meaning; direction carries it all. Mixing metrics across stages would mean the graph and the projection disagree about what “close” means.

Subtlety worth remembering: edges store cosine *similarity* (higher = closer), but NetworkX shortest-path routines read `weight` as *distance* (lower = closer). So `centrality.py` attaches a separate `distance = 1 - similarity` attribute and uses it for betweenness and closeness, while PageRank and strength use raw similarity. Getting this backwards silently inverts the meaning of every path-based metric.

4.5 PageRank over eigenvector centrality

Decision: PageRank ($\alpha = 0.85$) as the prestige metric.

Reasoning: the graph is disconnected — 13 components, 4 isolates. Eigenvector centrality collapses onto the giant component and crushes everything else to approximately zero. PageRank’s teleport term guarantees convergence and floors isolates at $(1 - \alpha)/N$. `eigenvector_diagnostic()` prints the evidence rather than asserting it.

Closeness uses the Wasserman–Faust correction (`wf_improved=True`) so that two-node islands stop out-ranking genuine hubs.

4.6 Threshold by target degree, not by intuition

Decision: the graph threshold τ is solved for, to hit a target mean degree of 8, rather than picked as a number.

Reasoning: a raw similarity threshold is uninterpretable across datasets. A target degree is a statement about the graph you want, and it transfers. Isolates are deliberately preserved — low affinity is signal, not an error.

5 Phase 1.5 — the pivot, and the load-bearing finding

Phase 1.5 was scoped as a one-day bridge to prove the synthetic pipeline worked on real data. It expanded substantially, because the real data invalidated the clustering approach outright. **This is the most important section of the document.**

5.1 What broke

Running HDBSCAN on the real 430-person network produced **336 of 371 titles as singletons**. There was no density to find. The parameter knob went dead above `min_cluster_size = 15`. A pre-registered gate of silhouette ≥ 0.35 at coverage $\geq 50\%$ failed outright.

The diagnosis: real LinkedIn titles are a long tail of near-unique strings. Synthetic data had clusters because the generator *built* clusters. Real data does not oblige.

5.2 The reframe

Occupation grouping is a classification problem, not a clustering problem. There is a real, externally-defined answer for each person — an occupation — and it should be looked up against a taxonomy, not discovered from the data’s internal geometry.

This replaced HDBSCAN with the O*NET Standard Occupational Classification, and replaced discovered cluster labels with the 23 SOC major groups.

5.3 The classification ceiling

Job titles alone support roughly **60–64% classifiability**. This is a property of the *input*, not of any method. It was measured three convergent ways:

1. **Inter-annotator agreement** between two human labellers on a blind 100-title set: **60%**.
2. **Self-inconsistency** — a single labeller disagreed with themselves on the same boundary cases the model failed on.
3. **Config E** scored **63.5%** precision on the blind 100.

Three humans and one model landing in the same band is the strongest available evidence that the remaining errors are undecidable rather than fixable. A two-word title frequently does not contain the information required to name an occupation.

The blind 100 is the only valid accuracy figure. The 50-title development set is contaminated — its labels were revised after predictions had been seen. Use it to choose between configurations and to find bugs. Never quote it.

5.4 The bake-off

`bakeoff.py` ran six configurations against the same labelled set, with a gate set *before* any run: precision $\geq 85\%$, coverage $\geq 50\%$.

Config	Description
A	Deterministic only (exact + fuzzy lexicon)
B	Deterministic + MiniLM embedding tier
C	Deterministic + Claude (title only)

Config	Description
D	Deterministic + Claude (title + company)
E	Claude only, title only
F	Claude only, title + company

Nothing cleared the gate — because the gate was set above the human ceiling. That is not a failure of the classifiers; it is a discovery about the problem.

Chosen: config E. Claude (claude-sonnet-5), title only, no company, no deterministic tier in front. Explicit abstention over fabrication.

Do not tune this further. The remaining errors are boundary titles that humans do not agree on either.

5.5 Ruled out permanently, with evidence

Approach	Evidence	Verdict
HDBSCAN clustering	336/371 singletons; no density; gate failed	Dead
MiniLM embedding classification	40% accurate; all five outputs scored 0.5517–0.5974, so good and bad were indistinguishable and no threshold could separate them	Net-negative
Company as a signal	Hurt precision across four independent runs	Rejected
Training on synthetic archetypes	Generator-inversion ceiling — recovers only our own mappings	Equivalent to a lookup table

The company finding is counter-intuitive and worth restating: company encodes *industry*, not *occupation*. A Financial Accountant at “Nalco Water, An Ecolab Company” gets dragged toward industrial chemicals. The signal is real but it points the wrong way.

The only lever that raises the ceiling is richer input — specifically, the user’s own “About” text, supplied as consented first-party data. That is P2.2.

5.6 Coverage-first with visible uncertainty

Decision: the ~39% of nodes the classifier abstains on render as grey “Needs review” nodes, visible and user-correctable. They are never hidden, and never labelled “Outlier”.

Reasoning: a confident wrong classification is the worst possible outcome, because it appears on a profile card citing an authoritative taxonomy and the user cannot detect that it is wrong. A flagged node is honest. The eval harness encodes this asymmetry explicitly — **missed** and **FABRICATED** both arise at the abstain boundary but are not equally bad, and thresholds are tuned to drive fabrication toward zero even at the cost of coverage.

This one decision generated the two load-bearing Phase 2 priorities (P2.1 and P2.2). They are not polish; they are the direct product consequence of the finding.

5.7 Scope limit: major group only

The classifier returns a **2-digit SOC major group**, not a detailed O*NET-SOC code. Asking for a detailed code invites invented codes — the model has no reliable index of the 1,016 valid ones.

This has an unresolved consequence. O*NET skills and tools join on *detailed* codes. Rendering taxonomy-derived skills on a profile card therefore requires a second matching stage that does not yet exist. See Section 10.

6 Decision log — Phase 2 (dashboard)

6.1 Selection is list-driven, not click-driven

Decision: the 3D graph is a pure *output* surface. Selection flows through the sidebar search/list panel.

Reasoning: Streamlit’s `on_select="rerun"` does not fire reliably on `Scatter3d`. The mechanism was built for 2D Cartesian plots; the bug tracker shows the same empty-selection behaviour across non-2D chart types. This was verified before building, not discovered afterwards.

The third-party `streamlit-plotly-events` component was rejected to keep the dependency surface minimal for Streamlit Community Cloud deployment.

And it is the better UX anyway. Precisely clicking one node in a 430-node 3D cloud with depth occlusion is impractical regardless of event support. The constraint pushed toward the design that was already correct.

Consequence: every DoD phrased as “clicking a node” must be re-worded to “selecting from the list”. A criterion that can never pass should be amended, not fudged at the QA checkpoint.

6.2 The correction overlay (`state.py`)

Decision: a flat `dict[int, str]` overlay keyed by `person_index`, applied eagerly and *uncached* on every rerun, returning a copy and never mutating the base frame.

It derives a `display_state` column with three values — `classified`, `needs_review`, `not_occupation` — which drives all colour, counter, and uncertainty logic. The `NOT_OCCUPATION` sentinel is kept strictly distinct from the “99” unreviewed-abstain code: “the user reviewed this and says it is not a job” is different information from “nobody has looked at this yet”.

Nine unit tests pass under plain `pytest` with no Streamlit runtime.

Why uncached: caching a correction overlay is how you get a UI that silently ignores the user’s last three corrections. See Section 7.

6.3 A fixed 23-colour palette

Decision: colours are assigned from `MAJOR_GROUP_NAMES` — the full SOC taxonomy — never derived from the groups present in any one network.

Reasoning: Version A means arbitrary user networks. If colour were derived from data, the same occupation would change colour between users, and any screenshot or documentation would be wrong for everyone but the person who made it. “Needs review” grey and “Not an occupation” grey are held *outside* the categorical hue space so they can never collide with a real group.

6.4 Faded-node highlighting

Decision: the figure splits into a matched trace at full opacity and an other trace at `FADED_OPACITY = 0.12`, using **scalar per-trace opacity**.

Reasoning: per-point RGBA alpha renders unreliably on `Scatter3d`. Two traces with scalar opacity is the mechanism that actually works. `hoverinfo="skip"` on the faded trace stops invisible nodes from stealing tooltips, and spikelines are disabled.

This mechanism is the reusable foundation for P2.6's cluster-dimming.

6.5 Edge overlay — built, then reverted

Decision: reverted. No similarity edges on the 3D canvas.

What happened: a top- $k = 3$ similarity overlay was built over 430 nodes, producing 978 edges. Rendered, it was a visual hairball — too densely connected to aid legibility at all.

Reasoning for the revert: this is informative signal about the density of the data, not a rendering failure. The network genuinely is that interconnected in embedding space. Displaying it does not help a user find anyone.

The similarity matrix is still cached and still powers centrality and the Phase 3 serendipity feature. It simply is not drawn.

6.6 Day-0 data reconciliation

Before any dashboard code, the canonical table was corrected at source:

- HDBSCAN-era column names renamed to honest taxonomy names: `cluster` → `soc_major`, `cluster_name` → `soc_major_name`.
- The abstain label "Outliers / bridge" corrected to "Needs review" in the pipeline, not in the UI. A misleading label fixed only at the display layer is still wrong in the data.
- Classifier provenance columns added.
- A coherent 3D UMAP projection generated as a sidecar, `data/cache/projected_3d.csv` (`person_index`, `umap_3d_x`, `umap_3d_y`, `umap_3d_z`). The main table retains `umap_x` / `umap_y`.

Security incident resolved during this session: a live Anthropic API key had been placed in `.env.example` — a file intended to be committed. Git history was confirmed clean, the key moved to `.env` (gitignored), and **the key rotated**. Rotation is the part that matters; moving the file alone would not have been sufficient.

7 Environment and tooling issues encountered

These cost real time. They are recorded so they cost nothing the second time.

7.1 The OneDrive migration

The project originally lived under OneDrive, whose file permissions and sync behaviour interfered with Claude Code. It was migrated to `C:\dev\network-visualiser` and the `.venv` rebuilt from scratch.

7.2 The wrong-venv trap

VS Code repeatedly activated an unrelated `onlyup` virtual environment.

Root cause: VS Code was opening `C:\dev` as the workspace root rather than `C:\dev\network-visualiser`, so the project's `.vscode/settings.json` was never read.

Fix: open the project folder *directly* as the workspace. Set the interpreter explicitly via `Python: Select Interpreter`. `.vscode/settings.json` pins `python.defaultInterpreterPath`, and `.vscode/` is gitignored because the path is machine-specific.

Standing rule: always `python -m pip`, never bare `pip`. Bare `pip` can resolve to a different environment than the active interpreter, installing packages where they will not be found.

Standing check before a session:

```
# Prompt should read: (.venv) C:\dev\network-visualiser>
python -c "import sys; print(sys.executable)"
```

The path must be inside the project venv.

7.3 The zombie Streamlit server

A Streamlit process left running on port 8501 continued serving a **stale module** while a newly-launched server ran on a different port. Code changes appeared to have no effect. Hours were lost to debugging code that was correct.

Diagnostic that worked: a standalone script (`diag_edges.py`) that built the figure entirely outside Streamlit. It confirmed the data pipeline was sound — 978 edges, correct dtypes, correct alignment — which isolated the fault to the server process.

Rule: when Streamlit behaviour contradicts the code, prove the data pipeline independently before touching the code.

7.4 The `@st.cache_data` trap

`@st.cache_data` locked in an empty edge CSV from *before* the file was restored. The cache outlived the problem it had captured.

Rule: anything that changes within a session — corrections, filters, derived state — must not be cached. This is why `state.py` applies the overlay uncached on every rerun.

7.5 The requirements.txt placeholder

A literal `plotly==<version-from-freeze>` had been pasted verbatim into `requirements.txt` instead of a real version number. Worth a glance before deployment at P2.9.

7.6 Plotly Scatter3d quirks

- `hovertemplate` and `hoverinfo` are **mutually exclusive**. Setting both silently drops one.
- Per-point RGBA alpha renders unreliably. Use scalar per-trace opacity.
- UMAP axes should stay hidden — the coordinates have no interpretable scale, and showing numbers implies a precision that does not exist.

7.7 Standing commands

Note — 22 September 2026 (D-21). The *Standing commands* table in this document is superseded by the *Development* section of `README.md`, which is kept current. It lacks the `python -m tools.<name>` form every tool needs, and its Git row (`git add .`) does not match how commits are made: explicit paths, reviewed with `git --no-pager diff --cached --stat`. The table below is left as written.

Purpose	Command
Run any <code>src/</code> module	<code>python -m src.<module></code> from repo root
Git	<code>git add .</code> → <code>git commit -m "..."</code> → <code>git push</code>
Diff without pager	<code>git --no-pager diff</code>
Search (PowerShell)	<code>Select-String / sls</code> — not <code>grep</code>
Find (PowerShell)	<code>Get-ChildItem -Recurse</code> — not <code>find</code>

The VS Code Run button fails with `ModuleNotFoundError` on `src/` files; the `-m` form is required.

8 Data artefacts and privacy posture

Path	Tracked?	Contents
data/graph/network_nodes.csv	Tracked	Canonical 430-row node table
data/cache/projected_3d.csv	Gitignored	3D UMAP sidecar
data/cache/embeddings.npy	Gitignored	384-dim vectors
data/cache/similarity.npy	Gitignored	Full cosine matrix
data/cache/claude_classification	Gitignored	Classifier cache
data/raw/Connections.csv	Gitignored	Real LinkedIn export
data/eval/	Gitignored	Labelled eval sets — personal data
.env	Gitignored	ANTHROPIC_API_KEY
.env.example	Tracked	Template — never a real key

8.1 Cross-cutting rules

- **Version A privacy:** real personal data processed ephemerally, never persisted server-side, never committed.
- **Honest representation:** no feature presents an uncertain classification, a similarity edge, or a centrality score as something stronger than it is.
- **Similarity, not social.** The graph is a *similarity* graph. Betweenness identifies bridges between skill areas — explicitly **not** “most socially connected”. Every insight that surfaces it must say so.
- **O*NET attribution** required in the README and the dashboard footer wherever taxonomy data is shown.

This system includes information from the O*NET 30.3 Database by the U.S. Department of Labor, Employment and Training Administration (USDOL/ETA), used under the CC BY 4.0 license. O*NET® is a trademark of USDOL/ETA. This project has modified some of that information. USDOL/ETA has not approved, endorsed, or tested these modifications.

9 Current board and what remains

9.1 Phase 2 — MVP dashboard (estimated 8–11 focused days)

Item	Status
P2.1 logic (<code>state.py</code> + 9 tests)	Done
P2.3 — 3D render, verified	Done
P2.4 + P2.4b — search panel + match-set highlight	Done
P2.5 — click-for-detail profile cards	Next
P2.1 UI — grey nodes + confirmation control	Pending — lands inside P2.5
P2.2 — self-enrichment via “About” text	Not started
P2.6 — SOC-tree cluster zoom, multi-select dimming	Not started
P2.7 — cached cluster summaries	Not started
P2.8 — chat interface wired to the Anthropic API	Not started
P2.9 — Streamlit Community Cloud deploy, README, demo	Not started

Approximately 3–4 days in; 2.5 of 9 items complete.

Flag: P2.1 and P2.2 are the two *load-bearing* priorities — the direct product consequence of the classification ceiling. One is half-built, the other untouched, and both currently sit behind foundation work that was meant to be secondary to them. P2.5 corrects half of this by absorbing P2.1’s UI.

9.2 Phase 3 — Team composition and network insight (5–7 days)

P3.1 team composition queries; P3.2 complementarity ranking; P3.3 serendipity suggestions via graph traversal; P3.4 top-connector chart from betweenness; P3.5 cluster skill matrices and co-founder pairings. All untouched, all dependent on a stable Phase 2 node table.

9.3 Phase 4 — Polish, narrative, public launch (4–5 days)

P4.1 team synergy narratives; P4.2 UI polish and dark mode; P4.3 PDF export; P4.4 onboarding tour; P4.5 performance documentation; P4.6 documentation and public launch with O*NET attribution and license compliance.

Remaining to public launch: approximately 14–19 focused days.

A QA checkpoint is signed off before any phase advances.

10 Open decisions and parked items

10.1 The 2D / 3D toggle — parked to Phase 4

Status: agreed as a product feature, deliberately deferred.

Reasoning: `build_figure` already isolates the coordinate columns, so the toggle is largely a matter of swapping which columns feed the scatter and switching `Scatter` for `Scatter3d`. That makes it cheap — but cheap later, not foundational now. It is a preference layer on top of a working canvas, so it belongs with dark mode and the onboarding tour in Phase 4 polish, not in the critical path.

Note for when it is built: the 2D coordinates (`umap_x`, `umap_y`) live in `network_nodes.csv` and the 3D coordinates live in the `projected_3d.csv` sidecar. These are **separate UMAP fits**, not a projection of one onto the other — so the two views will not correspond point-for-point in layout. That is methodologically correct but should be handled honestly in the UI rather than implying the views are the same picture rotated.

10.2 Taxonomy skills on profile cards — unresolved

The classifier returns a major group; O*NET skills join on detailed codes. Three options:

1. **Build the second matching stage now** — match within the winning major group to a detailed code. Half a day minimum, plus a fresh accuracy question, and it reopens Phase 1.5.
2. **Show group-level descriptors, labelled as group-level.** Honest and cheap, but low information.
3. **Ship no taxonomy skills section; populate skills only from P2.2 self-enrichment,** with the provenance tag the DoD already requires.

Recommendation: option 3, with a stub for option 2. Rendering specific skills against a person because their title landed in group 13 is exactly the confidently-wrong-citing-an-authoritative-taxonomy failure the whole abstention design exists to prevent. It would be individually-attributed data derived from a group-level inference, which the cross-cutting honesty rule forbids.

Do not resurrect `enrichment.csv` for this. Those archetype skills come from the superseded synthetic generator and only invert the project's own assumptions.

10.3 Persistent in-scene node label

Logged as a rider on P2.5: a name/role label on the selected node within the 3D scene. Recommended implementation is `layout.scene.annotations` rather than a text trace — scene annotations render in the 2D overlay layer and are therefore never occluded by nodes in front, which solves the depth problem outright rather than mitigating it.

10.4 Stage 2 productionisation — named, not built

Persistent database replacing CSVs; consent and onboarding flow; lawful basis and data-processing agreements for profiling; embedding versioning and a re-clustering schedule; authentication and per-organisation data isolation. Naming these in the write-up signals commercial maturity without building them.

11 The next task in full

P2.5 + P2.1 UI, combined. They are one task: the confirmation control has no surface to live on until the card exists, so building P2.1’s UI separately would mean inventing a container and rebuilding it inside P2.5 a day later.

11.1 Amended definition of done

1. Selecting any node from the list renders a card: name, raw title verbatim, assigned SOC group or “Needs review”.
2. Provenance is stated on the card — title-derived, user-corrected, or user-enriched.
3. A skills section is present but populated only from user enrichment; absent enrichment it states why, rather than presenting group-level data as personal.
4. Uncertain nodes additionally render the confirmation control: 23 SOC groups plus an explicit “not an occupation” option.
5. Assignment updates group, colour, and `display_state` on the next rerun, with no page reload.
6. Corrections persist in session state, never server-side.
7. “N of M nodes need review” decrements live, derived from the table.
8. The selected node carries a non-occluded name/role label in the 3D scene.
9. Inspection against `network_nodes.csv` confirms no uncertain node renders as confidently classified.

Estimate: 1–1.5 focused days.

Two DoD amendments are recorded against the original document — “clicking a node” becomes “selecting from the list” (Section 6.1), and the taxonomy-skills line becomes enrichment-sourced (Section 10.2). Both are honest rewrites of criteria that could not otherwise pass.

11.1.1 P3.11 — Disconnected professional pockets as a network insight

Logged 8 September 2026, Day 17, from a canvas diagnostic during the 442-row rebuild.

The far-right outlier on the 3D canvas is not an artefact. It is a tight cluster of five connections, all classified into the same group, all at identical degree centrality 0.013605 — connected to each other and to nothing else. UMAP places them far out because they are semantically distant from the rest of the network and highly similar to one another. The layout is correct.

Correction note — 6 October 2026 (D-78). Five names were removed from this entry.

The insight: the map can identify self-contained professional pockets with no bridge to the rest of a user’s network. That is a genuine finding about how someone’s career has been built, and it fell out of a two-minute diagnostic rather than a designed feature.

Candidate home: P3.4 (network insights / centrality chart), where it is a named example rather than an abstract metric, and P3.3 (serendipity), where “you know five people here and no one who connects them to anyone else” is exactly the suggestion the traversal should surface.

Rejected: percentile-clipping the canvas axes. Clipping would hide the clearest piece of structure on the map to tidy the middle, and a product built on honest representation must not silently drop a person from the view.

Phase 4 rider: an initial **scene.camera** framing the dense mass, letting the pocket sit at the edge of view. Presentation constant, not a data transform. Logged alongside dark mode.